are a fickle bunch, they might belong to more than one band at a time. To express this relationship in the database you need a dedicated table that associates a musician with a band. To associate a musician with multiple bands you need multiple rows that contain the musician's ID and each related band. You can see the difference between one-to-many and many-to-many relationships in figure 4.9.

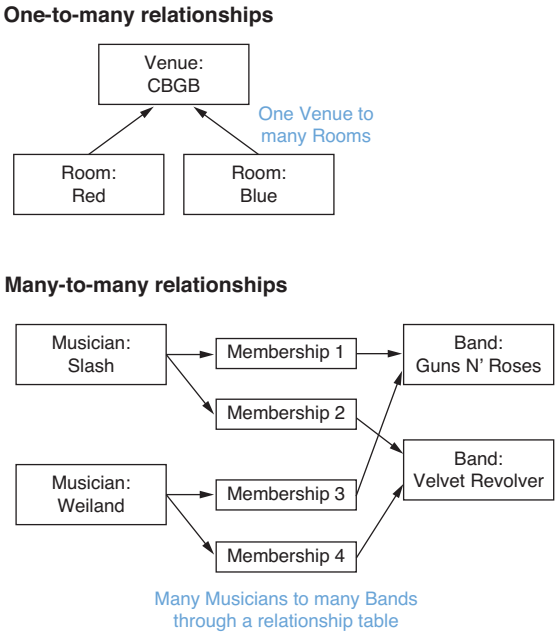


Figure 4.9 One-to-many vs. many-to-many relationships

Django supports both foreign-key and many-to-many relationships. It does this using `Model` fields named `ForeignKey` and `ManyToManyField`.

4.7.1 One-to-many relationships

A foreign-key relationship can feel a little backward. In writing, you might describe a venue containing multiple rooms. You're less likely to say there are multiple rooms all belonging to the same venue—it just doesn't flow as well. But that's how you do it in a database. This all comes down to which table contains the relationship column. If you put a *rooms* column on the venue table, you can only associate each venue with a single room. Remember, each row in the venue table is for a single venue. If instead, you put a *venue* column on the room table, each row describing a room can point at a single venue. This allows a venue to be associated with many rooms but a room only to be associated with a single venue.

Expressing a foreign key relationship in Django requires the addition of a field to your `Model`. The `ForeignKey` field requires at least two arguments. The first is a reference to the related `Model` object. This reference can either be the `Model` class itself or, optionally, a string naming the class. The string format is useful to avoid